

Using ML to Write Test Cases: A Literature Review

Lucas Swanson

CS 561 Research Assignment 7

Abstract

In this paper I have attempted to use different AI tools in order to collate and summarize different research papers on the use of AI in generating test cases. The paper is partly a focus in how to utilize AI to do this process quickly and easily, as well as an analysis of various studies within this field. The most prominent findings is that LLM's are very good at getting generic coverage of code, but are bad with hallucinations, targeted coverage, and keeping context of large codebases.

Comparison Table

Paper	Artifact type	Target / level	Model(s)	Augmentation / workflow	Dataset / benchmark	Evaluation focus	Notable limitation
Junwei Zhang, Xing Hu, 2026	Unit test generation	Function and class	GPT-family, LLaMA-based, encoder-decoder models	Prompt engineering, fine-tuning, agent-based approaches	Multiple studies across many benchmarks	Coverage, correctness, readability, compilability, naturalness	Review notes uneven datasets, languages, and metrics across studies
Ouedraogo, W.C., Kaboré, A.K., Li, Y., 2026	Unit test generation	Full class	GPT-3.5, GPT-4, Mistral 7B, Mixtral 8x7B	Zero-shot, few-shot, CoT, ToT, Guided ToT	Defects4J, SF110, CMD	Syntactic correctness, compilability, readability, coverage, test smells	Hallucinations still drive many compile failures
Wenhao Wang, Chenyuan Yang, Zhijie Wang, Yuheng Huang, Zhaoqiang Chiu, Da Song, Lingming Zhang, An Ran Chen, Lei Ma, 2025	Benchmark for test generation	Python programs / execution paths	16 commercial and open-source LLMs	Benchmark design; no generation method proposed	210 LeetCode Python programs	Overall coverage, targeted line/branch/path coverage	Targeted coverage remains hard for current LLMs
Roychowdhury, S., Sridhara, G., Raghavan, A.K., Bose, J., Mazumdar, S., Singh, H., Sugumaran, S. B., & Britto, R., 2024	Unit test generation	Java methods	LLM-based; specific model not emphasized in abstract	Static-program-analysis-guided prompting	One large commercial Java project; one open-source Java project	Effectiveness of test generation	Most useful when concise structural context is available
Pan, R., Kim, M., Krishna, R., Pavuluri, R., & Sinha, S., 2024	Multi-language unit test generation	Java and Python functions / classes	LLMs; specific models not foregrounded in abstract	Static analysis-guided pipeline	Standard Java apps, enterprise Java apps, large Python benchmark	Coverage, naturalness, developer preference	Needs static-analysis guidance to be competitive
Ben Hassine, S., & Wilson, S., 2024	ML test function generation	Machine learning source code	LLMs; specific model not emphasized in abstract	Representation-based comparison; retrieval-based baseline	Set of ML code and tests	Usefulness, documentation, correctness, programmer preference	Abstract gives little detail on scale and exact metrics
J. A. Nuh, T. W. Koh, B. Lawal, A. A. Jaharadak and M. A. Ali, 2025	Unit test generation	Python functions	GPT-style LLMs	Agent-based modular pipeline	Diverse Python functions	Correctness, passing/failing test generation	Abstract does not specify benchmark scale
Almeyda Alanía Fredy Antonio, Barrientos Padilla Alfredo and Siancas Garay Ronald Gustavo, 2025	Acceptance / BDD test generation	User stories to executable BDD tests	Dual-LLM engine (Gemini 1.5, GPT-4)	Schema-aware prompt orchestration, multi-level orchestration	50 user stories	Manual effort reduction, human quality rating, functional correctness	Focused on acceptance testing rather than unit tests
M. Y. Gundem and M. Karakose, 2026	Bug-driven test scenario generation	Bug descriptions to test scenarios	LLM-based; specific model not emphasized in abstract	Keyword extraction and structured prompt generation	Defects4J bug descriptions	Keyword match with error descriptions, scenario relevance	Appears narrower than full test-suite generation
M. Imane, H. Imad, M. Najib and M. Mohamed, 2025	Unit test generation	Text + focal methods to test cases	LLaMA3-8B	Prompt engineering; LLM-as-judge evaluation	Open-source projects; 136,060-entry dataset	Syntax correctness, test quality, generation scale	Abstract suggests quality still depends on combining text and code context

Discussion

This table shows ten research papers on different analyzing different methods and effective levels of analyzing and utilizing LLM's to assist a user in writing test cases. These papers all have slightly different methodologies for generating sets of test cases and have different focuses on how to compare each result. The methodology for creating test cases ranges from prompt engineering, pipeline generation-based, to keyword extraction. Evaluations focused on correctness (passing or failing), readability, coverage, test smells, and even the naturalness of use to a developer. Many different models were used, but most used the GPT-family or some LLaMA version. The targets of each paper were also wide and varied, all the way from whole Functions and Class coverage to Machine learning source code

generation. Now on to the critiques, starting with the first paper by Junwei Zhang (Zhang, 2026), and many others. Their paper tried to collate the results of around 115 different papers on unit test generation in order to compare the results and effectiveness of each. The different papers all had a widely varied dataset and were not initially intended to be compared against others, which can create a skewed database when making these large comparisons. In W.C. Ouédraogo's paper (W.C. 2026), a large empirical study looked at more than 200,000 auto-generated test cases, attempting to show that reasoning-based prompting guided by a "tree-of-thought" improves reliability and compatibility. However, this paper highlighted that even when guiding these LLM's, hallucinations still drive, not invalid test cases, but also compilation failures. In the paper written by Wenchang Wang (Wang, W, 2024), an attempt was made to benchmark 210 Python programs with three overall tasks: Overall coverage, targeted line/branch coverage, and targeted path coverage. This paper showcased that even across 16 LLM's used in the study, getting an LLM through prompts and entering the codebase in a query, getting targeted coverage is still difficult. In the Static Program Analysis paper (Roychowdhury, S, 2024), the group augmented prompts with concise context from program analysis to improve LLM unit test generation. It also validated the approach on a large commercial Java project and a popular open-source Java project. This improves upon the issue where getting targeted coverage is difficult, but it does not fix the underlying issue. Most of these studies or papers can improve the overall coverage with generated test cases, and can greatly improve developer efficiency. However, I think in order to move forward with these kinds of test generations in the industry, we need to address the key issues many of these face: hallucinations, lack of targeted coverage, and needing context when generating test cases. With these in mind, we can knowingly target test cases that work hand-in-hand with developers. LLM's that actively work in parallel while developers hand-write code may have a stronger ability to iterate on test cases and get coverage and context as the developer is working. I am curious to see how these prompt-based or pipeline-based test generation tools would compare to a developer writing test cases while using something like GitHub CoPilot, as it can gain the context of the tested code as the developer writes other test cases and can use more like predictive test case generation. This may lead to better performance overall in the area of LLM written test cases.

References Cited

- Zhang, J., Hu, X., Gao, C., Xia, X., & Li, S. (2026). *Enhancing automated unit test generation with large language models: A systematic literature review*. *ACM Transactions on Software Engineering and Methodology*.
<https://doi.org/10.1145/3802827>

- Ouédraogo, W.C., Kaboré, A.K., Li, Y. et al. Prompt engineering in LLMs for automated unit test generation: A large-scale study. *Empir Software Eng* **31**, 103 (2026). <https://doi.org/10.1007/s10664-026-10840-4>
- Wang, W., Yang, C., Wang, Z., Huang, Y., Chu, Z., Song, D., Zhang, L., Chen, A. R., & Ma, L. (2024). *TESTEVAL: Benchmarking large language models for test case generation*. arXiv. <https://doi.org/10.48550/arXiv.2406.04531>
- Roychowdhury, S., Sridhara, G., Raghavan, A. K., Bose, J., Mazumdar, S., Singh, H., Sugumaran, S. B., & Britto, R. (2024). *Static program analysis guided LLM based unit test generation*. In *Proceedings of the 8th International Conference on Data Science and Management of Data (12th ACM IKDD CODS and 30th COMAD)* (pp. 279–283). Association for Computing Machinery. <https://doi.org/10.1145/3703323.3703742>
- Pan, R., Kim, M., Krishna, R., Pavuluri, R., & Sinha, S. (2024). *Multi-language unit test generation using LLMs*. arXiv. <https://doi.org/10.48550/arXiv.2409.03093>
- Ben Hassine, S., & Wilson, S. (2024). *Representation and generation of machine learning test functions*. In N. Falk, S. Papi, & M. Zhang (Eds.), *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop* (pp. 238–247). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.eacl-srw.18>
- J. A. Nuh, T. W. Koh, B. Lawal, A. A. Jaharadak and M. A. Ali, "LLMTestCraft: An Automated Unit Test Case Generator using Large Language Models," *2025 IEEE International Conference on Data and Software Engineering (ICoDSE)*, Batam, Indonesia, 2025, pp. 331-336, doi: 10.1109/ICoDSE68111.2025.11351817. <https://ieeexplore.ieee.org/document/11351817>
- Almeyda Alania Fredy Antonio, Barrientos Padilla Alfredo and Siancas Garay Ronald Gustavo. "Engineering Prompt-Orchestrated LLM Workflows for Automated Test Case Generation in Agile Environments". *International Journal of Advanced Computer Science and Applications (ijacsa)* 16.12 (2025). <http://dx.doi.org/10.14569/IJACSA.2025.0161272>
- M. Y. Gundem and M. Karakose, "LLM Based Software Test Scenario Generation System From Bug Descriptions," *2026 30th International Conference on Information Technology (IT)*, Zabljak, Montenegro, 2026, pp. 1-4, doi: 10.1109/IT67293.2026.11435765.
keywords: {Software testing;Computer bugs;Software systems;Software;Software reliability;Scenario generation;Information technology;Artificial intelligence}. <https://ieeexplore.ieee.org/document/11435765>
- M. Imane, H. Imad, M. Najib and M. Mohamed, "A Novel Approach to Test Case Generation in Software Testing Using Pre-Trained Large Language Models," *2025*

12th International Conference on Wireless Networks and Mobile Communications (WINCOM), Riyadh, Saudi Arabia, 2025, pp. 1-7, doi:

10.1109/WINCOM65874.2025.11313396.

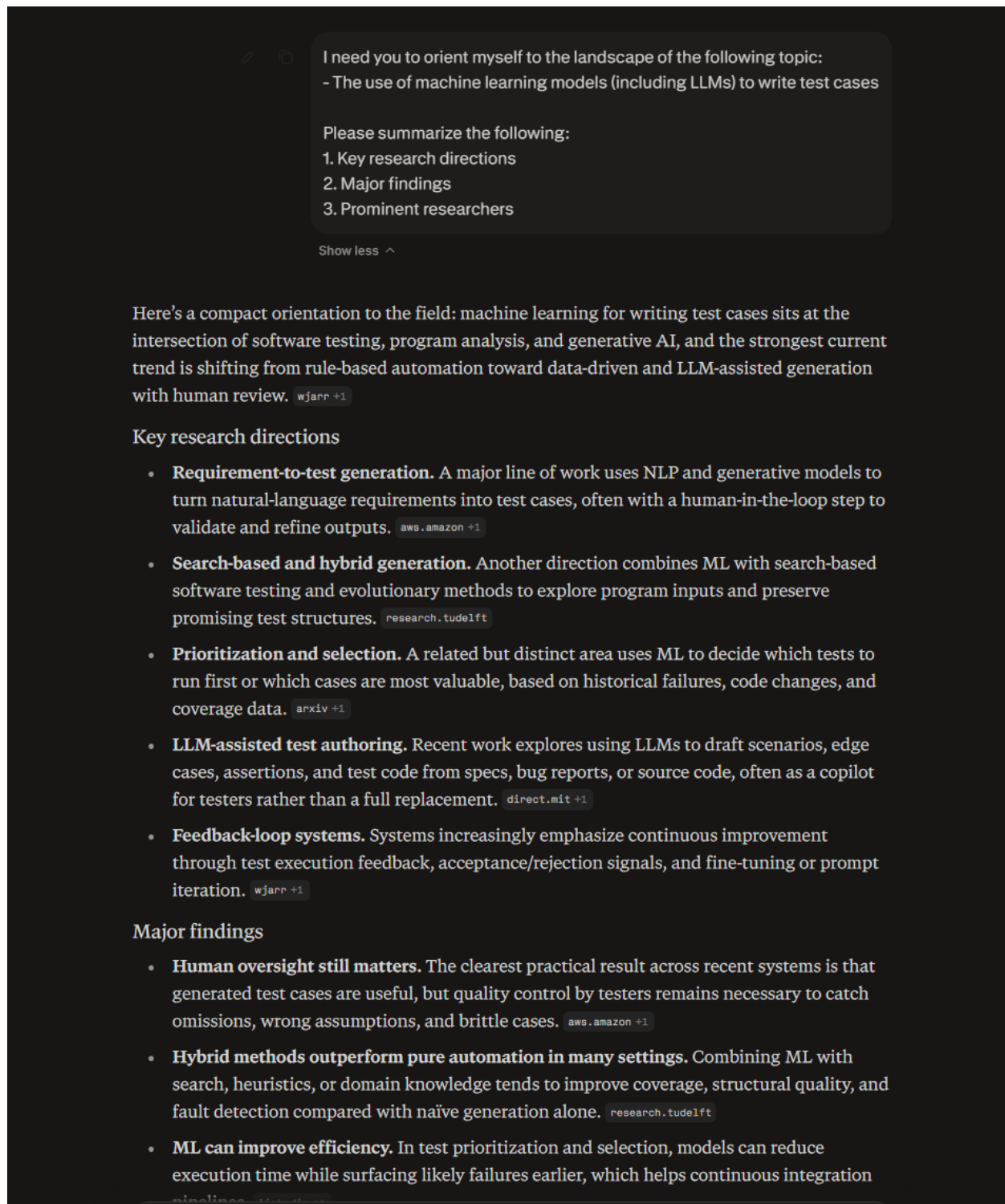
keywords: {Software testing;Codes;Large language models;Wireless networks;Natural languages;Syntactics;Mobile communication;Prompt engineering;Logic;Software engineering;LLaMA 3-8B;test case generation;software engineering;TDD;LLM as a judge},

<https://ieeexplore.ieee.org/document/11313396>

Research Process

In this research project, I chose a topic I did not know. Using AI to write test cases. The first thing I needed to do was understand the overall topic and outline key areas to focus on, such as whether the AI was using search-based results or hybrid generation, requirement-to-test generation of test cases, or whether the LLM would simply assist in test authoring. From these research directions, I chose to focus on one direction: LLM-assisted test authoring. After asking Perplexity to summarize this area of literature and picking a focus direction, I asked Elicit to “Help me find relevant research papers on the following topic: - LLM-Assisted Test Authoring. Please also list out some concept columns I can use to compare these together (e.g., model type, dataset, accuracy, etc.)”. This helped me find a large number of papers and easily distinguish each one based on what kind of artifacts were used in comparing results, what they focused on during evaluations, as well as any notable limitations with each. From here, I narrowed down to 10 research papers I thought were more organized and had less severe limitations.

Screenshot 1 — [Perplexity]



Here I asked PerplexityAI to give orient me to the landscape of writing test cases with LLM's, outline key research directions, major findings, and prominent researchers. It gave me

bullet point items in each category with a summary at the beginning. This helped outline where to begin and which categories of information I should be seeking out.

Screenshot 2 — *[Elicit]*

Can you help me find relevant research papers on the following topic:
- LLM-Assisted Test Authoring
Please also list out some concept columns I can use to compare these together (eg., model type, dataset, accuracy, etc)

I'll pull a quick set of papers on LLM-assisted test authoring, then I can organize them into a comparison table with useful columns like model, task, test type, and evaluation setup.

✓ Ran analysis

4 searches

- Q Academic · max 10 LLM test generation GitHub Actions unit tests paper
- Q Academic · max 10 automated test case generation LLM prompt-based testing
- Q Academic · max 10 large language model generated tests software engineering evaluation

🕒 Sources considered: LLM-Assisted Test Authoring

Search results loaded.

I found a concentrated cluster of papers on unit-test generation, plus a smaller set on acceptance-test/BDD generation and bug-driven scenario generation. The broader review I found says the literature already spans 116 papers, but it is still heavily concentrated on function- and class-level test generation, with less work on test inputs, oracles, and test evolution. [Zhang et al. 2026](#)

The most useful methodological pattern across the papers is grounding: static-analysis context, structured prompts, retrieval augmentation, and agent orchestration all try to make the generated tests more executable and less hallucinatory. In the largest empirical study I found, guided tree-of-thought improved reliability and compilability, but hallucination still caused compilation failures up to 86%. [Ouédraogo et al. 2026 +1](#) TESTEVAL is a strong benchmark paper to anchor comparisons because it uses 210 Python programs and separates overall coverage from targeted line/branch/path coverage. [Wang et al. 2024](#)

Two papers stand out if you want practical evidence rather than just a benchmark. The multi-language static-analysis pipeline reports coverage that is competitive with, and sometimes better than, state-of-the-art techniques, while also producing more natural tests; it also includes a user study with 161 professional developers. [Pan et al. 2024](#) The agile BDD paper reports 80% less manual design effort and 91.9% functional correctness on 50 user stories. [Antonio et al. 2025](#) A paper on ML test functions also found that programmers often preferred LLM-generated tests over retrieval-based alternatives, and sometimes over human-written ones. [Hassine & Wilson 2024](#)

Here I asked Elicit to find relevant research papers on this topic, and organize them with generic categories leaving it open for interpretation. It provided a list of 9 sources, with summaries and categories for each. This ended up being the baseline for my comparison table.

Screenshot 3 — [SciSpace]

The screenshot displays the SciSpace web interface. On the left is a sidebar with navigation options: Home, My Library, My Notebooks, Agent Gallery, AI Writer, Chat with PDF, Literature Review, Find Topics, Paraphraser, Citation Generator, Extract Data, and AI Detector. Below these are 'Recent Chats' with 'Deep Dive Research Paper' selected.

The main chat area shows a conversation with an AI agent. The user's message is: `https://link.springer.com/article/10.1007/s10664-026-10840-4` Can you help me dig deeper into this research paper?

The AI's response includes the following steps:

- Activating web-search-and-scrape skill**: I am activating the web-search-and-scrape skill to better handle the URL you provided and extract information from the research `web-search-and-scrape`.
- Skill Loaded**: `web-search-and-scrape`
- Scraping the research paper's webpage**: I am scraping the provided Springer Link URL to extract the paper's title, abstract, and other key details to begin our deep dive.

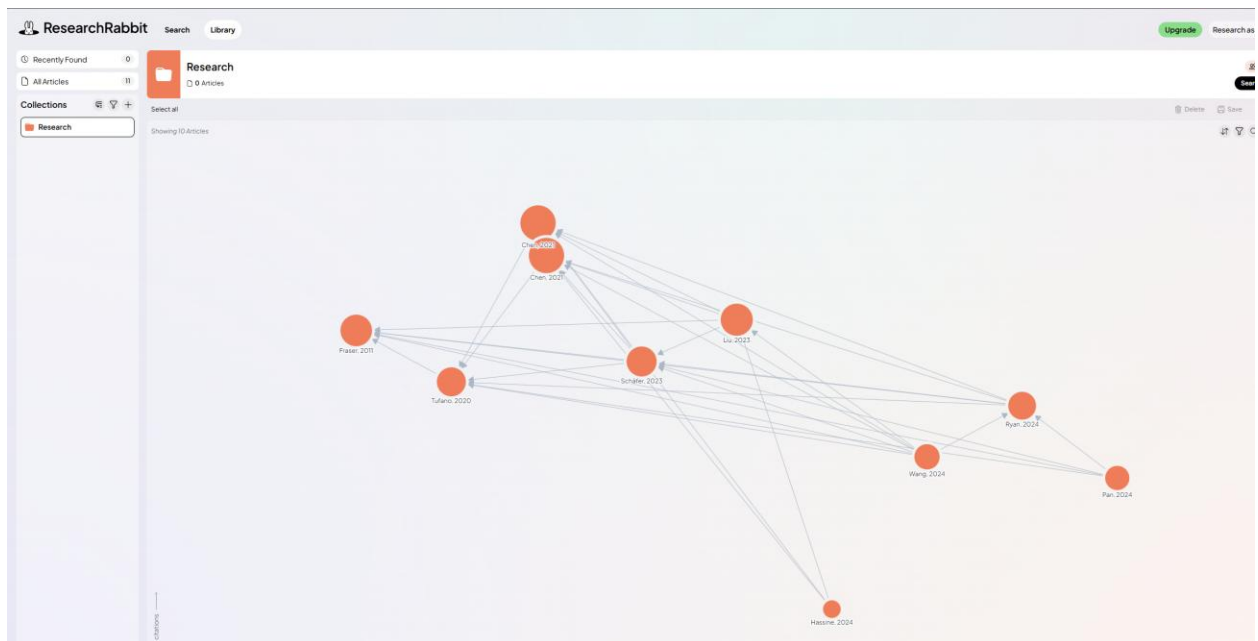
Below the text, a series of actions are shown in a scrollable list:

- `Read webpage`
- `Searched paper`: DOI: 10.1007/s10664-026-10840-4
- `Searched open access URL`: 10.1007/s10664-026-10840-4
- `Command`: `curl -L -o /home/sandbox/paper.pdf https://link.springer.com/content/pdf/10.1007/s10664-026-10840-4`
- `Command`: `file /home/sandbox/paper.pdf`
- `Command`: `head -c 100 /home/sandbox/paper.pdf`
- `Searched web`: "Prompt engineering in LLMs for automated unit test generation: A large-scale st..."
- `Read file`: `search_results.csv`
- `Command`: `curl -L -o /home/sandbox/axxiv_paper.pdf https://arxiv.org/pdf/2407.00225.pdf`

A 'Failed Command' notification is visible. At the bottom, a red banner states: 'Agent is on a coffee break. You have exhausted your available credits. To continue using SciSpace Agent, upgrade to a higher plan. Upgrade Now'.

Unfortunately, I tried a free account on SciSpace to assist me in deep diving a research paper, and it made so many queries itself that it locked me out of getting any results unless I paid \$20 for the base account with more credits. I had to leave this one out of the paper.

Screenshot 4 — [ResearchRabbit]



Here, I used research rabbit to expand upon my initial list of research papers from Elicit, and get more research papers into my search that relate and cite each other to get a better baseline of data from papers that revolve around similar topics with AI writing test cases.